

Intelligence Leading the Frontier, Computing Shaping the Future HIT Code Review Research

Experiment Report

Mining Code Review Comments and Understanding Requirements

Name	<u>Ulugbek Khamidov, Komil Tokhirov</u>	Department	<u>HITCodeReviewResearch</u>
Student ID	<u>2026140348, 2026140354</u>	Course Teacher	Liu Jiakun
Instructor	Liu Jiakun	Lab Location	<u>HIT First Campus</u>
Lab Time	<u>2024-06-11 14:00-17:00</u>		
Lab Score		Discussion Score	
Report Score		Total Experi- ment Score	

1. Experiment Objectives

- Understood PR and code review workflows in modern development. - Learned GitHub repository code review data structure. - Mastered relationships between PRs, commits, reviews, and comments. - Retrieved code review data via GitHub API. - Extracted code, text, and metadata from PRs. - Applied statistical analysis to build dataset for subsequent experiments.

2. Experiment Content

- Selected 5 projects: pytorch, tensorflow, kubernetes, vscode, langchain. - Retrieved 300 PRs per project (1,550 total after trimming). - Extracted PR metadata: ID, title, body, author, creation time, merge status. - Retrieved reviews: reviewer, decision, time, comment text. - Extracted commits: messages and diff content. - Collected labels from

each PR. - Detected AI reviewers using keyword matching. - Detected AI-generated code using keyword matching. - Performed statistical analysis on the full dataset.

3. Experimental Results

We collected 1,550 PRs from 5 open-source projects. Key statistics: - 813 merged (52.5%), 737 unmerged (47.5%). - Average additions: 9,110 lines. - Average deletions: 428 lines. - Average changed files: 88. - Average reviewers per PR: 1.8. - Average review comments per PR: 1.6. - AI reviewers present in 471 PRs (30.4%). - AI-generated code detected in 530 PRs (34.2%).

Six visualizations were produced showing merge status, review comments distribution, reviewers per PR, PR length, AI vs human code comparison, and top 10 labels.

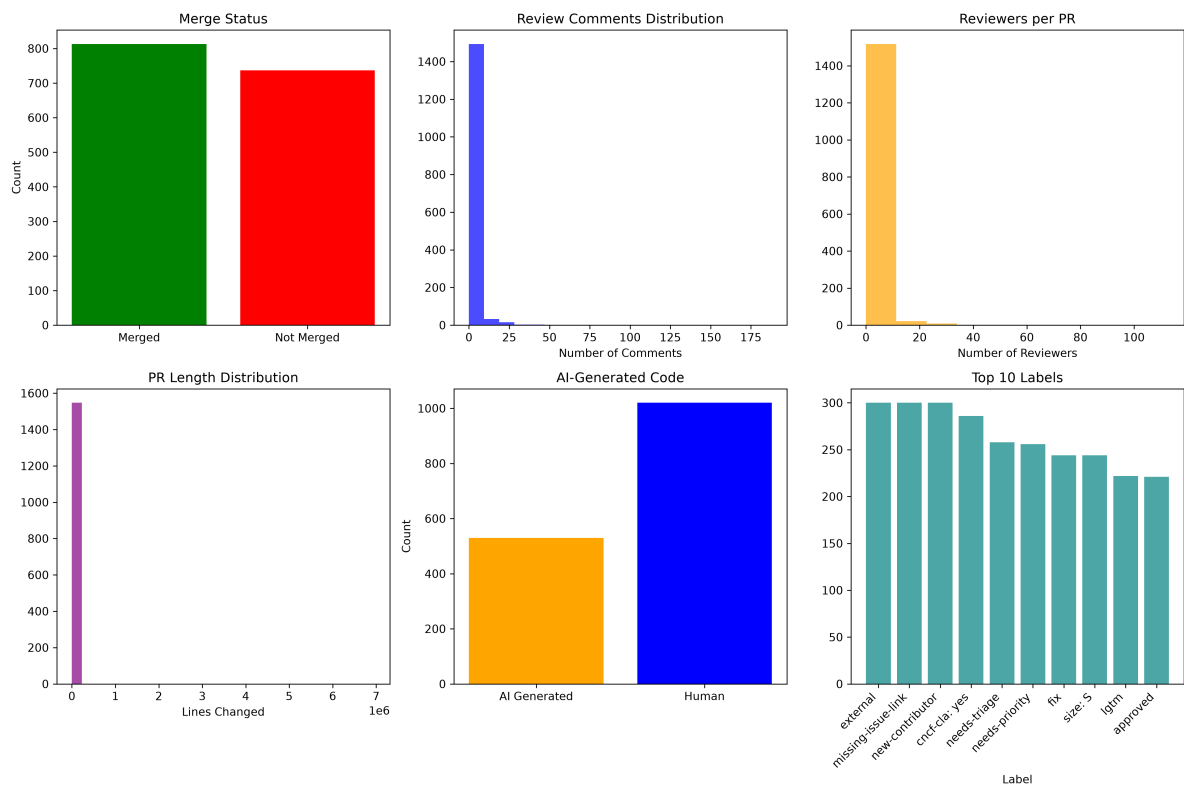


Figure 1: Statistical analysis of the PR dataset

4. Summary of Problems Encountered

- API rate limiting: added 0.15s delays between requests. - Network interruptions: implemented checkpoint saving every 5 PRs. - Langchain contained 9,450 PRs. Trimmed to 300 most recent for balanced distribution. - PyGithub deprecation warnings. Code functions correctly despite warnings.

5. Reflections on the Experiment

- PR is a merge request containing discussion and multiple commits. A commit is a single code change. - Code review catches defects early, enforces standards, and spreads knowledge across the team. - Merge prediction can flag high-risk PRs, prioritize reviews, and automate CI decisions. - AI reviewers respond instantly but lack project-specific context. Human reviewers understand broader design implications. - Our dataset has moderate imbalance (52.5% merged). SMOTE or class weighting can address this.

Instructor's Comments: Date: